

Priority-Aware Task Scheduling in Computing Power Network-Enabled Edge Computing Systems

Renchao Xie^{ID}, *Senior Member, IEEE*, Li Feng^{ID}, *Graduate Student Member, IEEE*, Qinqin Tang^{ID}, Han Zhu, Tao Huang^{ID}, *Senior Member, IEEE*, Ran Zhang^{ID}, *Member, IEEE*, F. Richard Yu^{ID}, *Fellow, IEEE*, and Zehui Xiong^{ID}, *Senior Member, IEEE*

Abstract—The Internet of everything, a potential direction for the next-generation Internet, positions edge collaboration as a promising computing paradigm to address the workload dispersion and resource constraints inherent in traditional edge computing frameworks. However, the increasing complexity of cross-domain networks introduces challenges for efficient task execution and balanced resource utilization in edge collaboration, which remain insufficiently explored. To address these challenges, a next-generation network architecture, the compute power network (CPN), was recently proposed. The CPN leverages ubiquitous connections among heterogeneous resources to optimize task scheduling collaboratively. Building on this concept, we design an edge computing system that integrates CPN to enable dynamic and collaborative task scheduling. Inspired by the sliding window, we develop a dynamic scheduling scheme that prioritizes computing tasks and matches tasks to computing resources in real time. Additionally, we propose an improved deep reinforcement learning (DRL) algorithm to optimize scheduling policies, aiming to improve task success rates, minimize execution delays, and ensure balanced and efficient resource utilization. Lastly, simulation experiments validate the effectiveness of the proposed scheme and algorithm.

Index Terms—Edge cooperation, computing power networks, scheduling priority, load-balancing, delay-sensitive.

I. INTRODUCTION

AS A supplement and extension of cloud computing, edge computing processes tasks at the network edge, close to users or data sources, offering low-delay, low-energy, and high-security intelligent computing services [1], [2], [3]. However,

the rapid advancement of information and communication technologies has introduced higher demands on network capabilities to support emerging computation-intensive and delay-sensitive applications [4]. While edge computing plays a pivotal role in improving computational and communication efficiency, its limited resource capacity poses significant challenges. Uneven task distribution across multiple edge nodes creates a natural demand for cooperative computing among nodes to better utilize idle resources within the edge network. Without such cooperation, uneven resource allocation could result in severe network congestion.

Based on this, edge collaboration has emerged as a strategic approach to pool resources across multiple edge computing nodes, alleviating the bottlenecks of isolated resources and improving the utilization of distributed resources. However, existing research on edge collaboration primarily focuses on task scheduling coordination within a limited range of neighboring computing nodes [5], [6], [7]. As networks expand, coordinating collaboration among edge nodes across different operators or domains remains a significant challenge that has yet to be thoroughly explored.

Fortunately, recent advancements in the computing power network (CPN) offer a promising solution [8], [9], [10], [11], [12], [13]. According to recommendation Y.2501 issued by the international telecommunication union (ITU), CPN is a novel network technology designed to optimize the distribution of computing, storage, and networking resources across computing nodes through a network control layer [8]. With its focus on on-demand task scheduling and flexible resource pooling, CPN presents itself as a strong candidate for enabling efficient edge collaboration in large-scale distributed edge systems.

As high concurrency becomes a critical consideration in the design of distributed system architectures, improving scheduling performance to enhance task processing efficiency and increase the success rate in CPN-enabled edge computing systems remains a key challenge [14], [15]. Task queuing is a widely adopted solution for managing high concurrency scenarios where the system's parallel processing capabilities fall short [16]. However, variations in task queuing positions can significantly impact the effectiveness of task processing. Moreover, in complex cross-domain systems, the diversity of tasks introduces additional challenges. Sequentially queuing tasks without accounting for their individual characteristics can reduce the success rate, particularly for delay-sensitive tasks. Assigning

Received 25 January 2024; revised 1 December 2024; accepted 31 March 2025. Date of publication 3 April 2025; date of current version 27 June 2025. This work was supported in part by the National Key R&D Program of China under Grant 2024YFB2906701 and in part by the National Natural Science Foundation of China under Grant 62401079. Recommended for acceptance by Dr. Yuan Wu. (Corresponding author: Qinqin Tang.)

Renchao Xie, Tao Huang, and Ran Zhang are with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing 100876, China, and also with Purple Mountain Laboratories, Nanjing 211111, China (e-mail: Renchao_xie@bupt.edu.cn; htiao@bupt.edu.cn; zhangran@bupt.edu.cn).

Li Feng, Qinqin Tang, and Han Zhu are with the State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: fengli@bupt.edu.cn; qqtang@bupt.edu.cn; zhhipu@bupt.edu.cn).

F. Richard Yu is with the School of Information Technology, Carleton University, Ottawa, ON K1S 5B6, Canada (e-mail: richard.yu@carleton.ca).

Zehui Xiong is with the Pillar of Information Systems Technology and Design, Singapore University of Technology and Design, Singapore 639798 (e-mail: zehui_xiong@sutd.edu.sg).

Digital Object Identifier 10.1109/TNSE.2025.3557385

different priorities to heterogeneous tasks based on their specific attributes presents a promising approach to addressing this issue.

To address efficient task scheduling in multi-node edge cooperation, this paper proposes a CPN-enabled edge computing framework and explores a priority-aware dynamic task scheduling scheme as a potential reference solution for high-concurrency scenarios. Our scheme adopts a sliding window approach to limit the number of tasks scheduled per time slot and evaluates scheduling priorities to improve the system's task processing success rate. From the service provider's perspective, the optimization goals include minimizing task delays, maximizing success rates, and ensuring efficient and balanced resource utilization. To adapt to the dynamic network environment, we propose a priority-aware deep reinforcement learning (PADRL) algorithm to develop strategic scheduling policies. The main contributions of our work are:

- *Proposed framework:* A CPN-enabled computation and networking integration framework is introduced to enable effective collaborative computing, along with a detailed description of the interaction procedures among various components.
- *Dynamic task scheduling:* To achieve optimal task-to-node matching based on real-time conditions, a dynamic task scheduling scheme inspired by the sliding window concept is designed. This approach queues tasks before scheduling and processes them immediately after being assigned to CPN nodes. Additionally, a priority-aware scheduling mechanism is proposed to enhance task execution efficiency.
- *Optimization algorithm:* A DRL-based algorithm, PADRL, is developed to optimize delay, success rate, and resource utilization performance, enabling the controller to make strategic scheduling policies.
- *Performance verification:* An ongoing scheduling process is simulated to validate the efficiency of the proposed approach, with numerical results demonstrating the superiority of the scheme.

The remainder of this paper is arranged in the following. Section II provides a review of related work on task scheduling in edge computing, edge collaboration, and CPNs. Section III introduces the detailed CPN-enabled framework, dynamic scheduling mechanism, and the system model. The DRL formulation is presented in Section IV. In Section V, we propose the PADRL algorithm to address the identified problem. In Section VI, we discuss the results of our simulations. The paper is brought to a conclusion in Section VII.

II. RELATED WORK

A. Edge Computing

Edge computing is acknowledged as one of the key pillars for satisfying the demanding performance indicators of the 5th generation (5G) mobile communication technology, especially in terms of low delay and bandwidth efficiency [17], [18]. Due to limited resources of a single node in edge systems, there have been many accumulations of research on task offloading or resource allocation related to edge computing in recent years.

In [19] and [20], the authors only consider the hierarchical offloading of a single terminal device, focusing on how to offload different tasks from their own devices to the cloud, the edge, or perform task processing locally to achieve the optimal system utility. The authors of [21] study the dynamic hierarchical task scheduling in the internet of vehicles and the complete task can be further divided into subtasks for offloading.

B. Edge Cooperation

However, the above studies mainly focus on task offloading in a specific domain, and the edge servers in charge of different regions cannot communicate or transmit data to each other, which will lead to a waste of idle computing resources of the edge layer. To overcome this challenge, edge cooperation is proposed to fully leverage resources. The authors of [22] establish a task offloading model based on multi-cloudlet cooperation, and an improved particle swarm optimization (PSO) algorithm is proposed to reduce the processing delay of different tasks and improve the utilization of cloud resources. In [23], the authors implement peer-to-peer (P2P) offloading between different edge servers with the support of software defined network (SDN) technology. When some servers in the system are overloaded, the average task processing delay can be minimized by jointly optimizing the offloading policy, routing, and computing resource allocation. Similarly, to relocate the tasks of a heavy-load edge server to its neighboring edge servers for processing, the authors of [5], [6], [7] further use the average field game or DRL methods to solve the task scheduling problem to achieve load balancing. The authors of [24] study the horizontal cooperation between edge nodes and the vertical task offloading between edge nodes and the remote cloud center. The queue status of different nodes is used to characterize the dynamic changes in the system.

C. The Computing Power Network

Based on computing offloading, edge cooperation further utilizes the idle computing resources of the edge layer. However, existing research related to edge cooperation mainly refers to cooperative task processing between adjacent nodes. With the ongoing expansion of network scope and the deep integration of computing and network, large-scale edge cooperation between multiple nodes across operators or domains is of great significance for improving the task handling and computational capabilities of the system, but it has not been fully studied. As a potential architecture for the forthcoming network generation, CPN can achieve flexible scheduling for large-scale edge cooperation, and it has garnered considerable interest within the academic community. In terms of development overview, the authors of [10] explore the evolving trajectory of computing and network convergence. In terms of performance optimization, the authors of [11] architect a unified framework on Internet protocol (IP) enhancement to achieve resource orchestration in CPN. What's more, to save energy, the authors of [12] suggest an anycast routing policy for CPN. Finally, in terms of system construction, the authors of [13] implement the prototype testbed of CPN based on Kubernetes with microservice architecture, and its performance is assessed using typical Internet services

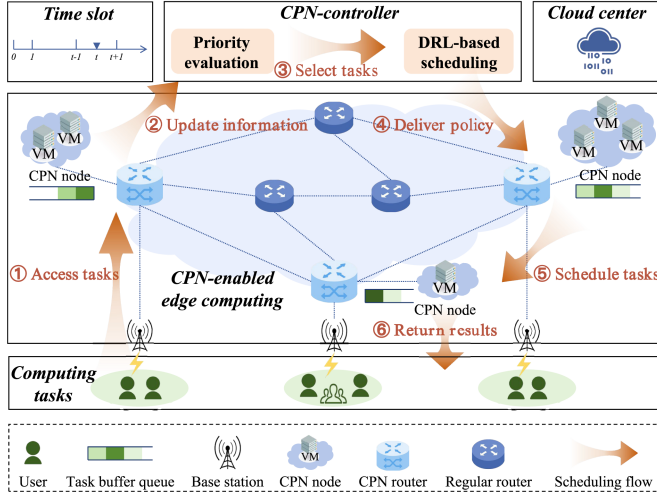


Fig. 1. Overview of the CPN-enabled edge computing system.

with intelligent inferences. The work conducted within the CPN framework enables enhanced orchestration control and resource scheduling for conventional edge computing. However, it is crucial to note that in practice, the key challenge lies in the CPN control plane's ability to flexibly schedule tasks according to specific requirements, ensuring resource sharing while maintaining system load balance.

III. SYSTEM DESCRIPTION

In this section, we introduce the system model and scheduling mechanisms in the CPN-enabled edge computing system, and describe the specific performance model.

A. The CPN-enabled Edge Computing System

As shown in Fig. 1, we propose a CPN-enabled edge computing system. The system primarily consists of resource demanders (computing tasks), resource providers (computing resources), and resource matcher (the CPN controller), with details as follows:

- **Computing tasks:** Tasks that require computing resources, generated by users, are collectively referred to as computing tasks. These computing tasks can connect to the base station from terminals and enter the network via the CPN router.
- **CPN nodes:** Heterogeneous computing resources are collectively referred to as CPN nodes. Each CPN node deploys virtual machine (VM) resources as the smallest computational unit, where each VM can only handle one task within a specific time slot.
- **CPN router:** Unlike regular routers, each CPN router has a task buffer queue that stores tasks accessed into the network and completes task forwarding based on the scheduling policy.
- **Cloud center:** While the system includes a cloud center to handle tasks beyond the edge's capabilities, we primarily focus on resource collaboration among edge nodes.

TABLE I
LIST OF NOTATIONS

Notation	Description
$\mathcal{E}, N, e_i$	Set, number and index of CPN nodes
$V_i, F_i, B_i(t)$	VM counts, computing capability and resource status of CPN node e_i
$\mathcal{R}, M, r_i$	Set, number and index of routers
l_{pq}	Link between CPN router r_p and r_q
$v_{l_{pq}}$	Real-time rate of link l_{pq}
$Q_i(t)$	Task buffer queue in CPN router r_i
L	Maximum capacity of the task buffer queue
$b_i^v(t)$	Resource status of VM v on CPN node e_i
$B_i^I(t), B_i^B(t)$	Count of idle and occupied VMs on CPN node e_i
$B^I(t), B^B(t)$	Count of total idle and occupied VMs
$k(t)$	Task queue identifier
$\tau_i^{k(t)}$	Task with $k(t)$ in $Q_i(t)$
$d_i^{k(t)}$	Scheduling action of $\tau_i^{k(t)}$
$s_i^{k(t)}, c_i^{k(t)}, a_i^{k(t)}, o_i^{k(t)}$	Input data volume, computing volume, delay requirement and arrival time of task $\tau_i^{k(t)}$
$\mathcal{T}_i^{k(t)}$	Total delay of task $\tau_i^{k(t)}$
$\mathcal{U}(t), \mathcal{D}(t), \mathcal{O}(t)$	Success rate, load balancing and idle overflow rate

- **CPN controller:** Based on the resource and task status within the system, the CPN controller formulates scheduling policies using optimization objectives and algorithms, aiming to optimize the overall performance of both resource demanders and providers.

Based on the aforementioned key components, we consider the scheduling of continuous tasks within the system and divide the total scheduling time into regular, fixed-duration slots, denoted by t , with each slot lasting T_0 . The collaborative task scheduling process in the system is as follows:

- **Step 1:** Computing tasks access the system via a CPN router and join the task buffer queue to await scheduling.
- **Step 2:** The CPN controller obtains updated task information and resource status from CPN nodes.
- **Step 3:** The CPN controller, combining system status and optimization mechanisms, formulates a scheduling policy.
- **Step 4:** The CPN controller dispatches the scheduling policy to various CPN routers.
- **Step 5:** CPN routers schedule tasks to the corresponding CPN nodes according to the policy.
- **Step 6:** Tasks are processed on the CPN nodes, and results are returned to the users.

B. System Model

In our system, we consider a network model primarily composed of CPN routers and CPN nodes, with the system's responsibility being to manage the scheduling of computing tasks within this network. The mathematical symbols used in our system are summarized in Table I.

1) **CPN Router:** Denote the set of all routers in the network as $\mathcal{R} = \{r_1, r_2, \dots, r_N, r_{N+1}, \dots, r_M\}$, where $\{r_1, r_2, \dots, r_N\}$ is the set of CPN routers, and $\{r_{N+1}, \dots, r_M\}$ is the set of regular routers. The CPN router serves as the gateway for task access and has a task buffer queue for storing tasks. Let $Q_i(t)$ denote

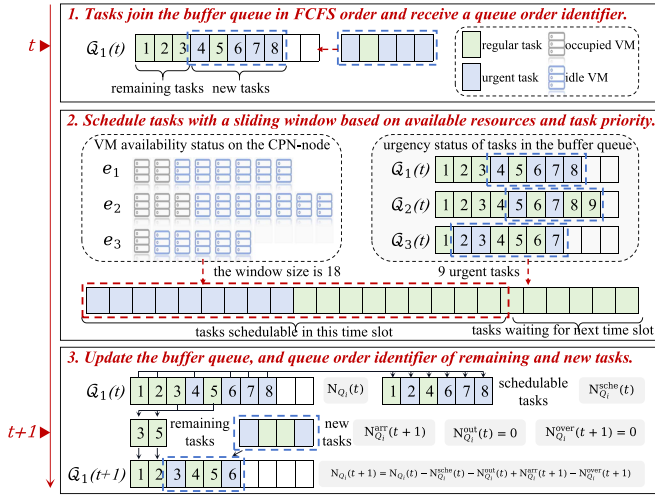


Fig. 2. Example of queue update process.

the buffer queue on CPN router r_i , which can accommodate up to L tasks. Denote the link between routers r_p and r_q as l_{pq} , where $p \in [1, M]$, $q \in [1, M]$, $p \neq q$, and the real-time rate of link l_{pq} as $v_{l_{pq}}$.

2) *CPN Node*: Denote the set of N CPN nodes in the system as $\mathcal{E} = \{e_1, e_2, \dots, e_i, \dots, e_N\}$, with each CPN node e_i directly connected to CPN router r_i . For CPN node e_i , its resource attributes are defined by the tuple $e_i = (V_i, F_i, B_i(t))$. V_i represents the number of VMs on the CPN node, corresponding to the number of tasks that can be processed in parallel. F_i is the maximum computing capability of the CPN node, which is evenly distributed among VMs. $B_i(t) = \{b_i^1(t), b_i^2(t), \dots, b_i^{V_i}(t)\}$ represents the resource status of the CPN node, where $b_i^v(t) \in B_i(t)$ indicates the resource status of VM v , with $b_i^v(t) = 1$ indicating that the resource is idle and ready to receive new tasks, whereas $b_i^v(t) = 0$ indicates that the resource is still occupied.

3) *Computing Task*: After tasks are generated at the terminal, they enter a task buffer queue upon reaching a CPN router, awaiting scheduling. Tasks can be directly scheduled to a CPN node connected to the CPN router or dispatched along the network to other CPN nodes for collaborative computing. Let the task that accesses CPN router r_i at time slot t and is positioned as the $k(t)$ order in the buffer queue be denoted as $\tau_i^{k(t)}$. The function $k(t)$, dependent on the time slot t , is used to indicate the order of a task in the buffer queue during a specific time slot, uniquely identifying each task. Each task $\tau_i^{k(t)}$ is defined by the tuple $(s_i^{k(t)}, c_i^{k(t)}, d_i^{k(t)}, o_i^{k(t)})$, representing the input data volume, estimated computational volume, delay requirement, and the time slot when it first entered the queue.

C. Scheduling Mechanism

Considering the complex state of the system, to ensure the timeliness of critical tasks and overall system efficiency, a priority-based task scheduling mechanism has been designed. This mechanism comprises two stages: determining the scheduling priorities of different tasks as shown in Fig. 2 and identifying the scheduling nodes for high-priority tasks.

Stage 1: Formulation of Scheduling Priorities:

The task buffer queue serves as a transitional area between the CPN nodes where tasks are accessed and exited, facilitating scheduling decisions. This dynamic queue operates in three phases: task joining, selection of schedulable tasks, and queue update.

1) *Task Joining*: Initially, tasks enter the queue in a first-come, first-served (FCFS) manner and await scheduling. To address differentiated delay requirements, tasks with delay constraints shorter than the scheduling slot duration are classified as urgent tasks, while others are designated as regular tasks. Upon entry, each task is sequentially assigned a unique queue identifier $k(t)$ based on the existing tasks in the queue.

2) *Selection of Schedulable Tasks*: The next step involves task priority planning. Using a sliding window approach and considering the remaining network resources, tasks in the buffer queue are scheduled equitably to minimize congestion and prevent CPN node overload. When network resources are insufficient to accommodate all tasks, a global priority must be established. Urgent tasks are assigned the highest priority and scheduled within the slot, redirecting to the cloud center if CPN node resources are inadequate. For regular tasks, priorities are determined using the algorithm outlined in Section V. High-priority tasks are then selected from the sorted queue for scheduling based on available system resources. Let the total number of idle VMs across all CPN nodes in the system be denoted by

$$B^I(t) = \sum_{j=1}^N B_j^I(t), \quad (1)$$

and the number of occupied VMs be

$$B^B(t) = \sum_{j=1}^N B_j^B(t) = \sum_{j=1}^N V_j - B^I(t). \quad (2)$$

Here, $B_j^I(t) = \sum_{v=1}^{V_j} b_j^v(t)$ and $B_j^B(t) = V_j - B_j^I(t)$ represents the number of available and occupied VMs on CPN node e_j . After global priority sorting, let the number of schedulable tasks in the buffer queue $Q_i(t)$ be denoted as $N_{Q_i}^{sche}(t)$. The total number of globally schedulable tasks should not exceed the number of available VMs, i.e.,

$$\sum_{i=1}^N N_{Q_i}^{sche}(t) \leq B^I(t). \quad (3)$$

3) *Queue update*: The process of task departure involves schedulable tasks being dequeued from the buffer queue and forwarded to the corresponding CPN node to obtain VM resources according to the scheduling policy outlined in Section V. Tasks that do not obtain scheduling opportunities and exceed their delay requirements in the task buffer queue are discarded. The remaining tasks in the queue wait for the next slot's sorting and window, and their identifiers are updated according to their new order in the queue. Additionally, new tasks will arrive in the next slot, denoting the number of new arrivals as $N_{Q_i}^{arr}(t+1)$. Thus, at the beginning of the next slot $t+1$, the total number of tasks

on the buffer queue can be calculated by

$$N_{Q_i}(t+1) = N_{Q_i}(t) - N_{Q_i}^{\text{sche}}(t) - N_{Q_i}^{\text{out}}(t) + N_{Q_i}^{\text{arr}}(t+1) - N_{Q_i}^{\text{over}}(t+1), \quad (4)$$

with $N_{Q_i}^{\text{out}}(t)$ representing the volume of tasks discarded due to timeout in the queue, and $N_{Q_i}^{\text{over}}(t+1)$ representing the volume of tasks from new arrivals that overflowed due to exceeding the queue length L . These tasks will immediately follow the existing remaining tasks to obtain corresponding identifiers.

Stage 2: Formulation of the Scheduling policy:

Once tasks eligible for a scheduling window are identified in each buffer queue, scheduling decisions must be made. Due to the large number of system nodes and high task access rates, employing a 0/1 variable planning model would result in significant decision space overhead. To address this, the decision action for task $\tau_i^{k(t)}$ is directly defined as its assigned scheduling CPN node, denoted by $a_i^{k(t)} \in [1, N]$. For instance, $a_i^{k(t)} = 3$ indicates that task $\tau_i^{k(t)}$ is scheduled to CPN node e_3 . From the service provider's perspective, collaborative scheduling decisions must be formulated globally for all schedulable tasks, based on the current network state and task requirements, represented as $A(t)$. These decisions aim to optimize overall system performance by maximizing task success rates, minimizing execution delay, and ensuring balanced and efficient resource utilization. The following sections will elaborate on these aspects.

D. Delay Model

The processing delay model can be categorized into two types based on the scheduling decisions. For computing tasks executed directly on the CPN node e_i connected to the accessed CPN router r_i , the cumulative delay comprises only the waiting and execution delays on that CPN node. In contrast, for tasks scheduled to collaborative CPN nodes e_j , the network transmission delay must also be accounted for. Additionally, the delay associated with transmitting results is typically negligible and is therefore disregarded, as commonly assumed in the related literature [21], [25].

1) *Transmission Delay*: The transmission delay of task $\tau_i^{k(t)}$ can be derived from the task size $s_i^{k(t)}$ and the transmission rates of corresponding routing links [26], [27], and it can be expressed as

$$R_i^{k(t)} = \sum_{l_{pq} \in \ell_{ij}} \frac{s_i^{k(t)}}{v_{l_{pq}}}, \quad (5)$$

where ℓ_{ij} represents the set of links the task passes through, and l_{pq} represents a specific link in this set. In the scheme of this paper, we use the shortest path algorithm, namely Floyd algorithm [28], to obtain the set of routing links between the ingress CPN router and the destination service nodes.

2) *Task Execution Delay*: In this paper, we assume that the total computing resources of a CPN node are equally distributed to each deployed VM. The task execution delay is determined based on the required computation volume $c_i^{k(t)}$ of this task, the

total computational capability F_j of the executed CPN node e_j , and the number of VMs V_j , that is,

$$C_i^{k(t)} = \frac{c_i^{k(t)}}{F_j/V_j}. \quad (6)$$

In summary, the total delay for task $\tau_i^{k(t)}$ can be denoted as

$$\mathcal{T}_i^{k(t)} = \begin{cases} C_i^{k(t)} + (t - o_i^{k(t)}), & a_i^{k(t)} = i \\ R_i^{k(t)} + C_i^{k(t)} + (t - o_i^{k(t)}), & a_i^{k(t)} \neq i. \end{cases} \quad (7)$$

Here, $o_i^{k(t)}$ represents the time slot when the task accesses the network, hence $t - o_i^{k(t)}$ denotes the waiting delay of the task in the buffer queue.

E. Success Rate Model

The success rate is a critical metric for evaluating the system's overall performance, accounting for two types of task failures: timeout discards and overflow discards.

1) *Timeout Discards*: It occurs when tasks are discarded because their time in the system exceeds their delay requirements. This can happen while tasks are still waiting in the buffer queue or after they have been processed but are no longer timely. Let $\mathcal{F}^{\text{time}}(t)$ represent the number of tasks discarded due to timeout within time slot t .

2) *Overflow Discards*: It occurs when tasks are discarded due to the system exceeding its memory capacity. This can happen in two scenarios: first, when the task buffer queue is full, preventing new tasks from entering, and second, when improper scheduling decisions lead to resource contention. For instance, if CPN node e_1 has only 3 free VMs but receives 4 tasks, while CPN node e_2 has 5 free VMs but only receives 4 tasks. Let $\mathcal{F}^{\text{mem}}(t)$ represent the number of tasks discarded due to overflow within time slot t .

Thus, the total number of failed tasks within time slot t is

$$\mathcal{F}(t) = \mathcal{F}^{\text{time}}(t) + \mathcal{F}^{\text{mem}}(t). \quad (8)$$

We can further obtain the success rate, i.e.,

$$\mathcal{U}(t) = \frac{\mathcal{G}(t)}{\Gamma}, \quad (9)$$

where Γ represents the average number of tasks accessed per time slot during the entire scheduling process, and $\mathcal{G}(t)$ denotes the number of tasks that are successfully executed on time during time slot t .

F. Resource Utilization Model

Resource utilization is another critical evaluation metric. In the system, users often prefer CPN nodes with abundant computing resources for task processing, which can lead to polarization: high-performance resources become overburdened, while low-performance resources remain underutilized or idle. This imbalance undermines the overall efficiency of the system. Consequently, this paper emphasizes load balancing of computational resources and addresses the issue of idle overflow rates to improve system performance.

1) *Load Balancing*: The load of CPN node e_j is represented by the utilization rate of the VM resources on it, i.e.,

$$w_j(t) = \frac{B_j^B(t) + \sum_{i=1}^N \sum_{k(t)=1}^{N_{Q_i}^{\text{sche}}(t)} \mathbb{T}\{a_i^{k(t)} = j\}}{V_j}. \quad (10)$$

Here, the number of occupied VMs includes those that were already in use at the beginning of the time slot, as well as those occupied by newly scheduled tasks. $\mathbb{T}(x)$ is an indicator function, which equals 1 when x is true.

Subsequently, the concept of variance from statistics is used to calculate the load balancing across all CPN nodes in the system, denoted as

$$\mathcal{D}(t) = \frac{\sum_{j=1}^N (w_j(t) - \bar{w})^2}{N}, \quad (11)$$

where $\bar{w}$ represents the average load across all CPN nodes.

2) *Idle Overflow Rate*: This is our newly proposed evaluation metric, which represents the ratio between the number of overflow tasks (i.e., tasks that could not be executed due to insufficient resources) and the number of idle VMs (i.e., VMs that were not assigned any tasks), denoted as

$$\mathcal{O}(t) = \frac{\mathcal{F}^{\text{mem}}(t)}{\sum_{j=1}^N V_j \cdot (1 - w_j(t))}. \quad (12)$$

This metric highlights the efficiency trade-off between task scheduling and resource allocation within the system. A high idle overflow rate indicates inefficiencies, as tasks remain unexecuted despite the availability of idle resources, often due to improper scheduling. Conversely, a low idle overflow rate reflects more effective resource allocation, where task execution is better aligned with VM utilization.

IV. DRL FRAMEWORK FORMULATION

In this section, we model the continuous scheduling process as a Markov decision process (MDP). This model can be represented as a tuple $\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}$, where $\mathcal{S}$ is the set of possible states, $\mathcal{A}$ is the set of actions, $\mathcal{P}$ represents the state transition probabilities, and $\mathcal{R}$ is the system reward function [29], [30], [31].

A. State

The whole system state $\mathcal{S}(t)$ is mainly composed of the state of tasks buffer queues, different routing links, and each CPN node, and it can be represented by

$$\mathcal{S}(t) = (\mathcal{Q}(t), \mathcal{L}(t), \mathcal{V}(t)). \quad (13)$$

$\mathcal{Q}(t)$ denotes the tasks buffer queue state of all CPN routers, and it can be represented as the following matrix when all the queues are full, i.e.,

$$\mathcal{Q}(t) = \begin{bmatrix} \tau_1^1 & \tau_1^2 & \cdots & \tau_1^L \\ \tau_2^1 & \tau_2^2 & \cdots & \tau_2^L \\ \vdots & \vdots & \ddots & \vdots \\ \tau_N^1 & \tau_N^2 & \cdots & \tau_N^L \end{bmatrix}, \quad (14)$$

where the row items represent task buffer queues on diverse CPN routers, and the column items identify distinct tasks within the

same queue. In particular, when the task queue is not full, $\tau_n^l = 0$ denotes that no task has come. Alternatively, it is distinguished by distinct task details $(s_i^{k(t)}, c_i^{k(t)}, d_i^{k(t)}, o_i^{k(t)})$.

$\mathcal{L}(t)$ indicates the current topological connection status of different routing links, and it can be expressed as

$$\mathcal{L}(t) = \begin{bmatrix} l_{11} & l_{12} & \cdots & l_{1M} \\ l_{21} & l_{22} & \cdots & l_{2M} \\ \vdots & \vdots & \ddots & \vdots \\ l_{M1} & l_{M2} & \cdots & l_{MM} \end{bmatrix}. \quad (15)$$

If there is a connection relationship between a pair of routers $(r_p, r_q), p \in [1, M], q \in [1, M], p \neq q$, l_{pq} can be represented by $v_{l_{pq}}$ as the data transmission rate of the link, otherwise $l_{pq} = 0$. To simplify the algorithm's solution process, we assume that the data transmission rate between links remains constant, while the connection status between nodes may vary.

$\mathcal{V}(t)$ expresses the condition of the diverse CPN nodes, including the occupied VMs situation $B_j^B(t)$ and the remaining task execution delay information $\mathcal{M}_j(t)$ of each occupied VM on CPN node e_j . This can be expressed as

$$\begin{aligned} \mathcal{V}(t) &= [V_{e_1}(t), \dots, V_{e_j}(t), \dots, V_{e_N}(t)] \\ &= [(B_1^B(t), \mathcal{M}_1(t)), \dots, (B_j^B(t), \mathcal{M}_j(t)), \\ &\quad \dots, (B_N^B(t), \mathcal{M}_N(t))] \end{aligned} \quad (16)$$

where $\mathcal{M}_j(t)$ is a set that characterizes the remaining task execution delay of every VM on e_j , this is further detailed by

$$\mathcal{M}_j = (t_1^{\text{remain}}(t), \dots, t_v^{\text{remain}}(t), \dots, t_{V_j}^{\text{remain}}(t)), \quad (17)$$

where V_j is the total number of VMs on CPN node e_j . In this paper, the remaining processing delay of a VM is represented by the number of fixed-length scheduling time slots, simplifying the calculation. For instance, if a task requires two time slots to complete, the remaining processing delay of the occupied VM is expressed as $t_v^{\text{remain}}(t) = 2$.

B. Action

For task $\tau_i^{k(t)}$, the scheduling action is denoted by $a_i^{k(t)}$. Thus, the action for globally schedulable tasks is also represented as

$$A(t) = [A_1(t), \dots, A_i(t), \dots, A_N(t)]^T. \quad (18)$$

The scheduling action must satisfy the following constraint: the number of tasks scheduled to a CPN node e_j should not exceed the number of idle VMs on it, denoted as

$$\sum_{i=1}^N \sum_{k(t)=1}^{N_{Q_i}^{\text{sche}}(t)} \mathbb{T}\{a_i^{k(t)} = j\} \leq B_j^I(t), \forall j \in [1, N]. \quad (19)$$

If the action decision does not satisfy this constraint, tasks exceeding the number of idle VMs will overflow and fail.

C. State Updating

Based on the scheduling actions taken, the state of the system will be updated accordingly.

For the buffer queue state $\mathcal{Q}(t)$, tasks that have been scheduled or failed will leave the queue, and new tasks will access it. The tasks on the buffer queue will be updated according to (4).

For the link state, we assume that the link rates remain constant, but the connection status between links may change, which can be updated based on periodic network monitoring.

For the CPN node state, on the one hand, the usage status of VMs will be updated by

$$B_j^B(t+1) = B_j^B(t) + \sum_{i=1}^N \sum_{k(t)=1}^{N_{Q_i}^{\text{sche}}(t)} \mathbb{T}\{a_i^{k(t)} = j\} - \sum_{v=1}^{V_j} \mathbb{T}\{t_v^{\text{remain}}(t) = 1\}, \forall j \in [1, N]. \quad (20)$$

On the other hand, the remaining task execution delays will also be updated by

$$t_v^{\text{remain}}(t+1) = \max\{t_v^{\text{remain}}(t) - 1, 0\}, \\ \forall t_v^{\text{remain}}(t+1) \in \mathcal{M}_j, \forall j \in [1, N]. \quad (21)$$

D. Reward

As outlined earlier, this study seeks to improve the task success rate while minimizing the average delay of successfully executed tasks. Additionally, it aims to reduce the load imbalance and idle overflow rate, promoting balanced and efficient resource utilization. Consequently, the reward for scheduling policies within each scheduling slot t can be expressed as

$$\mathcal{R}(t) = \mu_1 \mathcal{U}(t) - \mu_2 \frac{1}{\mathcal{G}(t)} \sum_{i=1}^N \sum_{k(t)=1}^{N_{Q_i}^{\text{sche}}(t)} \mathcal{T}_i^{k(t)} - \mu_3 \mathcal{D}(t) - \mu_4 \mathcal{O}(t), \quad (22)$$

where μ_1 , μ_2 , μ_3 and μ_4 represent the weighting factors [32], used to control and adjust the impact of different factors on the final scheduling policy.

Based on this, the objective of task scheduling is to determine a policy π that maximizes the reward associated with the decision.

V. THE PROPOSED PADRL ALGORITHM

In this section, we propose the PADRL algorithm to address the dynamic scheduling problem. The PADRL algorithm is composed of two main components: selecting tasks with higher scheduling priorities based on a priority assessment mechanism, detailed in Algorithm 1, and matching these high-priority schedulable tasks with suitable CPN nodes, as described in Algorithm 2.

A. Algorithm 1: The Priority Evaluation Algorithm

To determine the scheduling priority of each task, different priority evaluation criteria may be generated to meet various scene requirements. In this paper, a priority evaluation scheme that comprehensively considers multiple influencing factors is

Algorithm 1: Priority Evaluation Algorithm Based on Multiple Factors.

Input: The number of total idle VMs $B^I(t)$. The scheduling time slot length T_0 . Status of tasks arriving at each ingress router.

Output: Scheduling priority of each task.

```

1 Initialize auxiliary parameter  $N = 0$ .
2 for each task  $\tau_i^{k(t)}$  of the system do
3   if the remaining delay budget  $d_i^{k(t)} \leq T_0$  then
4     set the scheduling priority to 1
5      $N = N + 1$ 
6   end
7   else if the remaining delay budget  $d_i^{k(t)} > T_0$  then
8     Calculate the priority score  $p^{\text{score}}(\tau_i^{k(t)})$  according to Eq. 23
9   end
10 end
11 Sort tasks with remaining delay budget greater than  $T_0$  by priority score
12 Select the top  $(B^I(t) - N)$  tasks to form a set  $G$ 
13 for tasks with remaining delay budget  $d_i^{k(t)} > T_0$  do
14   if task  $\tau_i^{k(t)} \in G$  then
15     set the scheduling priority to 1
16   end
17   else if task  $\tau_i^{k(t)} \notin G$  then
18     set the scheduling priority to 0
19   end
20 end

```

proposed, which is summarized as shown in Algorithm 1. The specific prioritization process is described below.

In our system, the CPN controller gathers node state and task information to enable centralized and unified scheduling. During each scheduling slot, tasks with delay requirements shorter than the system's scheduling time slot, i.e., urgent tasks, are prioritized to ensure their successful processing. These tasks are assigned the highest scheduling priority, with their priority value set to 1. If the number of such tasks exceeds the available idle VMs in the system, the surplus tasks are redirected to the cloud center for processing. However, as this paper primarily focuses on task scheduling within the edge layer, the handling of tasks in the cloud center is beyond the scope of this study and is not further explored.

For tasks with delay requirements longer than the scheduled time slot, i.e., regular tasks, determining their priority involves a comprehensive evaluation of multiple factors. First and foremost, the delay requirements of tasks remain a critical factor that must be considered. Second, considering the queuing status of the task buffer queue at each CPN router, task scheduling priority can be influenced by the load at each ingress CPN router, as calculated by (4). Specifically, the more tasks queued at a CPN router, the higher the priority assigned to scheduling tasks from that router during the current time slot. Last, tasks arriving in the system at earlier slots should be given higher scheduling priority, reflecting their longer wait time. In summary, for tasks whose delay requirements exceed the scheduling time slot length, their priority must account for the task's delay requirements, the ingress router's task load, and the task's arrival order in the

system. To ensure a fair evaluation, each task is assigned a score based on these three aspects, and priority is determined according to the task's score ranking. As an example, taking task $\tau_i^{k(t)}$ as follows,

$$p^{\text{score}}(\tau_i^{k(t)}) = \beta_1 (-d_i^{k(t)}) + \beta_2 \cdot N_{Q_i}(t) + \beta_3 (T_e - o_i^{k(t)}), \quad (23)$$

where T_e represents the arrival time slot of the earliest arriving task of the system in this slot.

According to the first term in (23), the score increases as the task's delay requirement becomes stricter, prioritizing tasks with lower tolerable delays. The second term highlights that tasks arriving at routers with heavier loads receive higher scores, reflecting the system's preference for balancing workload distribution. The third term indicates that tasks entering the system earlier are assigned higher scores, emphasizing the importance of processing tasks promptly. Additionally, the coefficients β_1 , β_2 , and β_3 can be adjusted flexibly to suit specific scenarios or requirements, allowing the scoring mechanism to adapt to different system priorities.

Finally, for these two types of tasks, urgent tasks are always assigned the highest priority, while the remaining tasks are sorted in descending order of priority. Upon the arrival of the sliding window, the system schedules tasks in order of priority based on the currently available resources.

B. Algorithm 2: The TD3 Algorithm

Reinforcement learning (RL) is a powerful approach for solving MDP, enabling the discovery of optimal strategies through interactive trial-and-error learning to maximize cumulative rewards. However, traditional RL faces limitations in high-dimensional state and action spaces due to the curse of dimensionality. DRL addresses these challenges by leveraging neural networks to approximate value or policy functions, significantly expanding RL's applicability. Methods such as deep Q-network (DQN) [33] mitigate storage pressures, while algorithms like proximal policy optimization (PPO) [34] and soft actor-critic (SAC) [35] are well-suited for continuous action spaces. DRL excels in dynamic scenarios by learning directly from interactions without requiring explicit state transition models.

DRL methods are categorized into value-based (e.g., DQN), policy optimization-based (e.g., PPO, SAC), and hybrid actor-critic algorithms (e.g., asynchronous advantage actor-critic (A3C) [36], twin delayed deep deterministic policy gradient (TD3) [37]). Among these, SAC and TD3 are commonly used for continuous action spaces. SAC emphasizes exploratory behavior and balances exploration and exploitation, while TD3 reduces overestimation bias using dual Q-values and delayed policy updates, enhancing stability and accuracy. TD3 is particularly effective for continuous scheduling tasks requiring precision and stability, and its computational efficiency makes it ideal for resource-constrained environments. Studies show that TD3 outperforms SAC in task completion, delay optimization, and energy efficiency, aligning with the objectives of this paper [38],

[39], [40], [41], [42], [43]. Therefore, TD3 is selected as the solution.

TD3, derived from deep deterministic policy gradient (DDPG) [44], addresses value function overestimation through three key innovations:

- *Dual Q-value mechanism*: Two independent Q-networks estimate the value function, and the lower estimate is used as the target value, mitigating overestimation.
- *Delayed policy updates*: Policy networks are updated less frequently, reducing reliance on inaccurate value estimates and improving optimization stability.
- *Target policy smoothing*: Adding noise to target actions decreases dependency on precise predictions.

These features make TD3 an effective and reliable choice for the optimization objectives considered in this paper.

In TD3, the neural network is central to optimizing the task scheduling policy. Its network update process includes three phases: initialization, exploration, and update.

- *Initialization*: Construct two independent Q-networks, Q_{θ_1} and Q_{θ_2} , a policy network Π_{ϕ} , and their corresponding target networks $Q_{\theta'_1}$, $Q_{\theta'_2}$, and $\Pi_{\phi'}$.
- *Exploration*: At each time step, the algorithm interacts with the environment to collect information, including the current state s_t , action a_t , reward r_t , and next state s_{t+1} . These experiences (s_t, a_t, r_t, s_{t+1}) are then stored in an experience replay buffer, which is subsequently used to train the neural networks, enhancing learning stability and efficiency.
- *Update*: The TD3 algorithm optimizes the value and policy functions by sampling experiences from the experience replay buffer. The target Q-values are computed using TD3's dual Q-value mechanism, which applies the temporal difference (TD) method for approximation. This method avoids the infinite summation challenge in the theoretical definition of the value function. The target Q-value is calculated as

$$y = r + \gamma \min_{i=1,2} \{Q_{\theta'_i}(s', \tilde{a})\}, \quad (24)$$

where $\tilde{a} = \Pi_{\phi'}(s') + \epsilon$ introduces noise ϵ for target policy smoothing, and γ is the discount factor. This mechanism enhances stability and reduces overestimation in Q-value predictions.

Both Q-networks are updated by minimizing the mean squared error loss function

$$L(\theta_i) = \frac{1}{N} \sum_{j=1}^N (y_j - Q_{\theta_i}(s_j, a_j))^2, \quad i \in \{1, 2\}. \quad (25)$$

The policy network is optimized by maximizing the Q-values, but it is updated only once during delayed updates, which is as follows:

$$L(\phi) = -\frac{1}{N} \sum_{j=1}^N Q_{\theta_1}(s_j, \pi_{\phi}(s_j)). \quad (26)$$

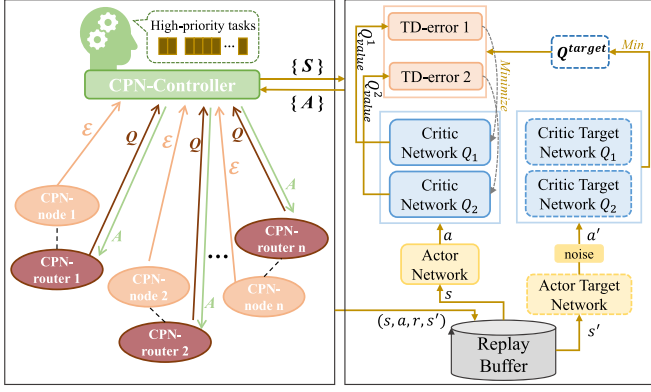


Fig. 3. The framework of the proposed PADRL algorithm.

The target networks are updated using a soft update mechanism, ensuring gradual alignment with the primary networks and enhancing training stability. This process is defined as

$$\theta'_i \leftarrow \tau \theta_i + (1 - \tau) \theta'_i, \quad \phi' \leftarrow \tau \phi + (1 - \tau) \phi', \quad (27)$$

where $\tau \in (0, 1)$ is the soft update coefficient that controls the rate of parameter blending, ensuring smooth transitions and stable learning dynamics.

The core idea of this network update method is to decouple the frequency of policy learning from value function estimation. By doing so, it ensures the reliability of the value function estimates while maintaining control over the quality of policy updates. This separation contributes to a stable and efficient training process, reducing the risk of overfitting or instability in policy optimization.

C. The PATD3 Algorithm

The PATD3 algorithm integrates a priority assessment mechanism with the TD3 algorithm to efficiently address task scheduling. As shown in Fig. 3, which is summarized in Algorithm 2, the CPN controller trains the neural network through multiple episodes, each consisting of several steps corresponding to scheduling slots. In each slot, tasks are prioritized based on urgency and importance, ensuring critical tasks are scheduled first. The TD3 algorithm then assigns the selected tasks to specific CPN nodes for efficient allocation. As scheduling progresses, the system state updates continuously, enabling the CPN controller to refine decisions. Once converged, the controller implements policies based on the trained model for adaptive task scheduling.

To solve scalability issues in high-dimensional action spaces, PATD3 has made key modifications. In traditional scheduling problems, actions are represented by an N-dimensional boolean vector that indicates all possible node selections. As the number of nodes increases, the dimensionality of the action space expands linearly, significantly increasing computational complexity. Therefore, PATD3 simplifies the action representation to a target node identifier, i.e., $a_i^{k(t)} \in \{1, 2, \dots, N\}$, reducing the action space from a multi-dimensional boolean vector to a single scalar, thereby significantly lowering computational complexity. Moreover, as the algorithm uses a continuous action

Algorithm 2: The proposed PADRL Algorithm.

Input: The state of arriving tasks, VMs, and different routing links.

Output: The scheduling policies for different tasks: $\{a_i^{k(t)}\}$.

```

1 Initialize critic networks  $Q_{\theta_1}, Q_{\theta_2}$  and actor network  $\Pi_{\phi}$ .
2 Initialize target network  $Q_{\theta'_1}, Q_{\theta'_2}$  and  $\Pi_{\phi'}$ .
3 for episode = 1,  $M$  do
4   Set up a stochastic process  $\mathcal{N}$  to explore actions randomly.
5   Select the tasks with high scheduling priority based on Algorithm 1.
6   Receive observation state  $s_1$ , including the state of high-priority tasks, VMs on CPN nodes, and routing links.
7   for  $t = 1, T$  do
8     Choose an action with added exploration noise
        $a_t = \pi_{\phi}(s_t) + \mathcal{N}$ .
9     Obtain current reward  $r_t$  and next state  $s_{t+1}$ .
10    Save the transition tuple  $(s_t, a_t, r_t, s_{t+1})$  in the memory buffer R.
11    Retrieve a mini-batch of  $B$  transitions  $(s_t, a_t, r_t, s_{t+1})$  from the memory buffer R.
12    Select  $\tilde{a}$  by  $\tilde{a} = \Pi_{\phi'}(s') + \epsilon$ .
13    Calculate  $y$  as described in Eq. 24.
14    if  $t \bmod p$  then
15      Update the policy network according to Eq. 26.
16      Update the target networks according to Eq. 27.
17    end
18    Update Q-networks as described in Eq. 25.
19  end
20 end

```

space ranging from $(-1, 1)$, it requires a mapping function to convert the continuous action values provided by the algorithm, a_i^{cont} , into discrete node identifiers, i.e.,

$$a_i^{k(t)} = \left\lfloor \frac{a_i^{\text{cont}} + 1}{2} \cdot N \right\rfloor + 1. \quad (28)$$

This mapping accommodates the flexibility of continuous action outputs and the need for discrete task scheduling, effectively enhancing the algorithm's computational efficiency and scalability in high-dimensional dynamic environments.

Simultaneously, to enhance sample utilization efficiency, the algorithm employs a prioritized experience replay mechanism [45]. This approach assigns priorities to experience samples based on their TD error, allowing samples with higher learning value to be revisited more frequently, thereby accelerating policy optimization. Specifically, the priority of each sample is determined by its TD error, denoted as

$$\delta_i = |y_i - Q_{\theta}(s_i, a_i)|. \quad (29)$$

The probability of sampling a given experience is proportional to its priority

$$P_i = \frac{\delta_i^{\alpha}}{\sum_j \delta_j^{\alpha}}, \quad (30)$$

where α is a hyperparameter that controls the impact level of the priority. By prioritizing critical samples, the PATD3 algorithm ensures faster and more effective policy optimization, significantly improving learning efficiency.

TABLE II
LIST OF PARAMETERS

Parameter	Value
Total number of CPN nodes	(10,20)
Number of VMs on each CPN node	(5,20)
Computing power of each CPN node	(100,1000) GCPU/s
Length of task buffer queue	15
Link transmission rate	(100,500) MB/s
Required computing amount per task	(1,50) GCPU
Input data per task	(1,20) MB
Proportion of urgent tasks	0.3
Actor learning rate	1e-4
Critic learning rate	3e-4
Target network update rate	0.01
Batch size	128
Discount	0.95
Explore noise	0.05
Policy noise	0.25
Noise clip	0.5

These enhancements streamline the action space, significantly improving the algorithm's sample utilization efficiency and providing robust support for deploying PATD3 in high-dimensional dynamic environments. By integrating action space dimensionality reduction with prioritized experience replay, PATD3 effectively addresses the complexities of task scheduling demands. This approach ensures improved adaptability, efficient resource allocation, and sustained convergence and stability under dynamic conditions.

VI. SIMULATION RESULTS

In this section, we evaluate the effectiveness of our proposed priority-aware task scheduling scheme in supporting CPN-based edge computing systems using numerical results.

A. Simulation Settings

We assume a network topology of 10 to 20 CPN nodes and routers to simulate realistic scenarios. Each CPN node randomly hosts 5 to 20 VMs, with total computational power ranging from 100 to 1000 GCPU/s and a task buffer queue length of 15. Link transmission rates range from 100 to 500 MB/s. The simulation spans 60 scheduling slots, each lasting 1 second, representing a continuous scheduling process. Tasks with computational demands distributed uniformly arrive at an average rate of 10 tasks per second per CPN node, requiring 1 to 50 GCPU and 1 to 20 MB of input data. Approximately 30% of the tasks are urgent.

The proposed algorithm was implemented in PyTorch using Python 3.9, with optimal hyperparameters determined through extensive testing (detailed in Table II). To validate the superiority of the PATD3 algorithm, we compare it against three advanced DRL algorithms (TD3, DDPG, and SAC) and two baseline approaches: the Local algorithm, which prioritizes local computing, and the Rand algorithm, which assigns tasks randomly.

B. Convergence Tests

Prior to applying the algorithm, we conducted offline training of the DRL neural network in the test environment and evaluated its convergence performance under various neural network hyperparameter settings. These results were compared with several

baseline algorithms. Each test group underwent training over 500 or 1000 episodes using 5 random seeds, with performance metrics averaged across the 5 training environments for analysis.

Fig. 4 evaluates three critical parameters that influence the algorithm's convergence: learning rate (LR), batch size (BS), and policy noise (PN). Fig. 4(a) analyzes the impact of the learning rate, which controls the step size for parameter updates during training. An excessively high learning rate ($LR = 1e-3$) leads to oscillations or divergence, causing the algorithm to miss the optimal solution. Conversely, an excessively low learning rate ($LR = 1e-6$) slows convergence significantly, requiring more training epochs to reach stability. A moderate learning rate ($LR = 1e-4$) achieves a good balance between stability and convergence speed, making it the optimal choice for this algorithm. Fig. 4(b) examines the influence of batch size, which determines the number of samples used for gradient updates. A large batch size ($BS = 256$) accelerates each training iteration but risks converging to suboptimal solutions due to reduced gradient variability. On the other hand, a small batch size ($BS = 32$) introduces higher noise and oscillations in the training process due to frequent updates. A moderate batch size ($BS = 128$) provides a balance, ensuring stable and efficient convergence while maintaining training consistency. Fig. 4(c) evaluates the effect of policy noise, a mechanism that introduces random perturbations to stabilize policy updates and prevent overfitting or getting trapped in local optima. Excessive noise ($PN = 0.55$) causes output actions to deviate beyond effective ranges, hindering stable learning. In contrast, insufficient noise ($PN = 0.1$) fails to adequately address overfitting, reducing the algorithm's adaptability. An optimal noise level ($PN = 0.25$) strikes a balance between exploration and stability, ensuring effective and reliable policy learning. Together, these results highlight the importance of fine-tuning key parameters to achieve robust algorithm performance and convergence.

Fig. 5 compares the convergence performance of four DRL algorithms under varying CPN node scales, with each algorithm utilizing the optimal hyperparameters identified in prior tests. In Fig. 5(a), with a node scale of 10, all algorithms exhibit relatively good convergence. The proposed PATD3 algorithm converges quickly, stabilizing around 100 episodes, and maintains consistent performance. In contrast, TD3, which lacks a priority planning mechanism, struggles to prioritize high-priority tasks among those schedulable, leading to significant learning difficulties and noticeable oscillations. DDPG encounters overestimation issues, resulting in large fluctuations and a lower final convergence value. SAC, although starting with a higher initial reward due to noise-free exploration, converges prematurely to a local optimum, stabilizing after approximately 60 episodes and failing to explore further improvements. Fig. 5(b) evaluates the algorithms with a CPN node scale of 15. While PATD3 continues to perform well, achieving stable convergence after approximately 600 episodes, TD3 and DDPG still experience significant oscillations, reflecting persistent learning challenges in larger-scale scenarios. SAC remains in an exploratory phase, showing little improvement in reward values as the training progresses. When the node scale increases to 20, as shown in Fig. 5(c), these trends become even more pronounced. PATD3

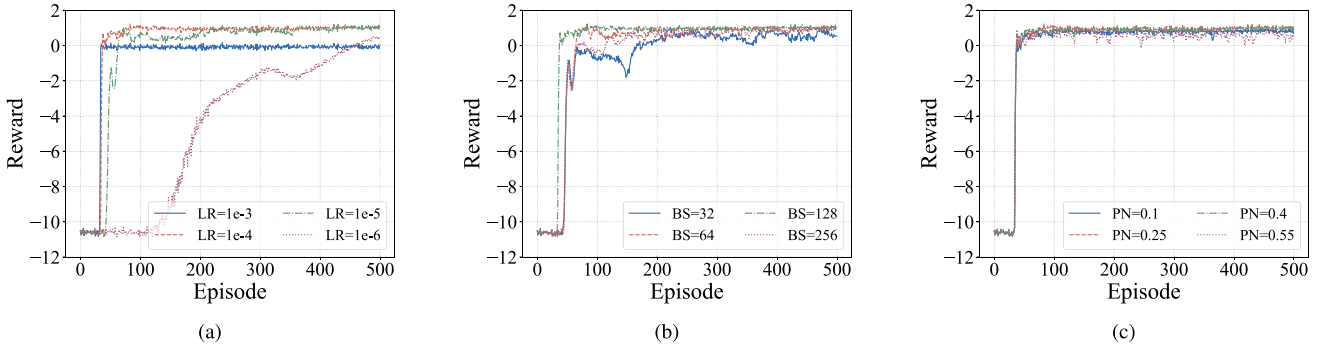


Fig. 4. Convergence performance of PATD3 with different: (a) learning rate; (b) batch size; (c) policy noise.

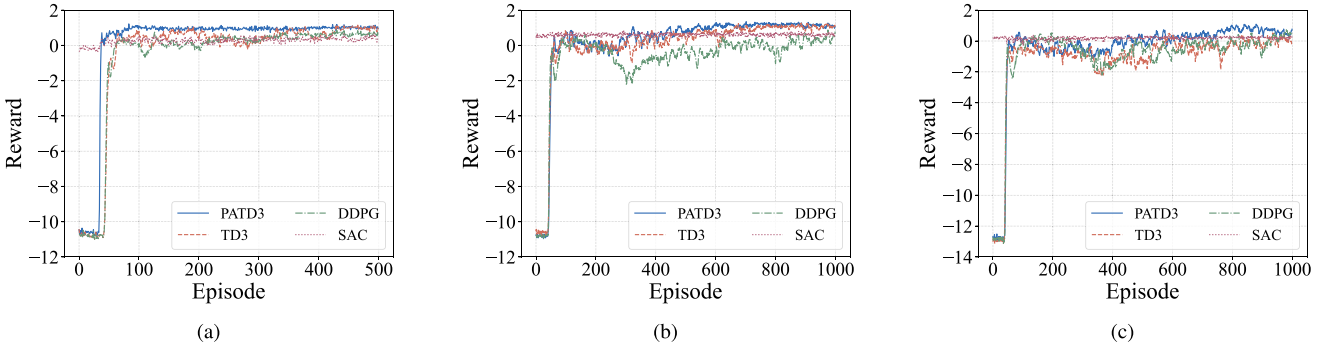


Fig. 5. Convergence performance of DRL-based algorithm under different CPN node scales: (a) 10; (b) 20; (c) 30.

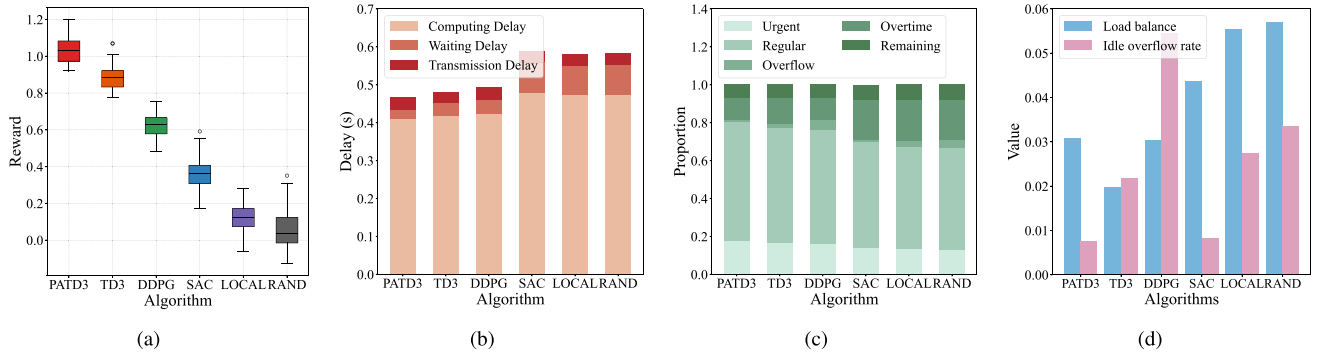


Fig. 6. Performance indicators of different algorithms: (a) reward; (b) delay; (c) success rate; (d) resource utilization.

continues to demonstrate robust convergence, while the performance of the other algorithms deteriorates further due to the increased complexity of task scheduling in larger networks. To ensure reliable and consistent results, we adopt a CPN node scale of 10 for subsequent comparative analyses, balancing the complexity of the simulation with the need for algorithm evaluation under realistic conditions.

C. Performance Analysis

After completing offline training for each algorithm, the best-performing model of each type was selected for real-time testing and analysis under identical initial conditions. Each algorithm was tested 100 times to ensure robust evaluation.

Fig. 6(a) presents the reward values from the 100 tests using box plots. The results clearly show that PATD3 consistently achieves the highest reward values with no outliers, reflecting its superior stability and performance. In contrast, TD3, lacking a priority planning mechanism, produces lower reward values and includes one outlier, highlighting its limited optimization capability. DDPG demonstrates even lower convergence values with significant oscillations, leading to further reductions in reward performance. SAC yields lower reward values as well, with a taller box height and an outlier, indicating poor stability in this scenario. The LOCAL algorithm, constrained to executing tasks only on accessed nodes, experiences resource congestion, particularly on low-resource nodes, severely limiting its performance. Although the RAND algorithm incorporates collaboration, its

lack of an optimization strategy results in poor stability and low reward values. In summary, PATD3 outperforms TD3 by 15.8% in overall performance and surpasses the RAND algorithm by 160.1%, demonstrating its effectiveness and efficiency in task scheduling.

In addition to evaluating overall rewards, we further analyze the individual components of reward optimization to gain deeper insights into system performance.

Fig. 6(b) presents the delay component of reward, including computation, waiting, and transmission delays, visualized as a stacked bar chart. The results indicate that the delays for the PATD3, TD3, and DDPG algorithms remain below 0.5 seconds. Among these, PATD3 demonstrates significant optimization in both computation and transmission delays, a result of its priority planning mechanism, which ensures that critical tasks are processed more efficiently. While PATD3 experiences a slight increase in transmission delay, its total delay is still the lowest among all algorithms, highlighting its effectiveness in reducing overall task completion time. In contrast, the delays for the remaining three algorithms exceed 0.5 seconds, with SAC showing the highest delay. This suggests that SAC does not adequately prioritize delay minimization within its optimization framework, leading to suboptimal performance in delay-sensitive scenarios. These findings underscore PATD3's superior ability to handle tasks with stringent delay requirements, outperforming other algorithms in achieving low delay and efficient resource utilization.

Fig. 6(c) illustrates the proportions of different task execution outcomes, represented in stacked bars. From bottom to top, each bar shows the proportions of successfully executed urgent tasks, successfully executed regular tasks, overflowed tasks, overdue tasks, and remaining tasks (unscheduled or still in execution). PATD3 outperforms all other algorithms by achieving the highest proportions of successfully executed urgent and regular tasks, with negligible overflowed tasks. Its ability to prioritize urgent tasks ensures minimal overdue and remaining tasks, reflecting its effective scheduling strategy. In contrast, the TD3 algorithm experiences a significant increase in overflowed tasks, primarily due to the lack of a priority planning mechanism, which delays scheduling decisions and raises the risk of queue overflows. This issue is further exacerbated in DDPG, resulting in even poorer task execution outcomes. Although SAC manages to reduce the number of overflowed tasks, it suffers from a lower success rate for both urgent and regular tasks. Furthermore, the proportion of overdue tasks remains relatively high, indicating suboptimal performance in delay-sensitive scenarios. The two non-optimized algorithms, LOCAL and RAND, perform the worst, with low success rates and higher proportions of overdue and overflowed tasks, underscoring their inefficiency in managing task execution under dynamic conditions.

Fig. 6(d) compares load balancing and idle overflow rate, two critical metrics for evaluating resource utilization. While PATD3 does not achieve the lowest load balancing value, it demonstrates superior optimization of the idle overflow rate, effectively minimizing resource wastage. This highlights its ability to make efficient use of available resources even under dynamic conditions. TD3 achieves the lowest load balancing

value, showcasing its emphasis on evenly distributing resource loads across nodes. However, DDPG exhibits an excessively high idle overflow rate, indicating a significant mismatch between available idle resources and the task overflow demands, leading to inefficient utilization. SAC achieves a relatively low idle overflow rate but performs worse than PATD3 in terms of load balancing, underscoring its limitations in handling resource distribution effectively. The two non-optimized algorithms fare the worst, with both high load balancing values and high idle overflow rates, reflecting suboptimal resource utilization and poor adaptability to varying task demands.

D. Adaptability Evaluation

In addition to evaluating the algorithms' applicability under different network scales (Fig. 5), we further tested their performance across scenarios with varying task loads, computational capabilities, and task types.

Fig. 7(a) presents the reward values of the algorithms under different task loads. Task load is defined as the average number of tasks accessed per second per CPN node, with the test range increasing from an average of 6 tasks to 14 tasks. The results indicate that as task load increases, overall reward values show a clear decline. This decline can be attributed to two main factors. First, as the number of tasks increases while node computational capabilities remain constant, the overall task success rate decreases, accompanied by a rise in overflowed or timed-out tasks. Second, the larger task volume significantly expands the action space dimensions, increasing the complexity of the optimization process. Consequently, the algorithms experience reduced optimization efficiency and effectiveness. The combined impact of these factors contributes to the observed decrease in overall performance, highlighting the challenges posed by high task loads in maintaining system efficiency and algorithmic robustness.

Fig. 7(b) illustrates the variation in reward values across different levels of computing power. To simulate diverse computational environments, we adjusted the computing power of each CPN node, with the test range increasing from an average computing power of 300 GCPU/s to 700 GCPU/s. The results show that as computing power increases, task computational delays are significantly reduced, which in turn lowers waiting delays and contributes to a substantial decrease in overall delays. Faster task execution not only enhances system responsiveness but also promotes the reuse of computational resources, effectively expanding the scheduling window for each time slot. This leads to an improvement in task success rates. Moreover, the reduction in task processing time decreases the load values of individual nodes, resulting in lower load balancing values, which further optimizes resource distribution. The combined effects of reduced delays, increased resource reuse, and improved load balancing drive a steady increase in reward values as computing power improves, highlighting the critical role of computational capacity in enhancing system performance.

Finally, we evaluated the algorithm's applicability in delay-sensitive scenarios by varying the proportion of urgent tasks among all tasks to simulate different application scenarios. The

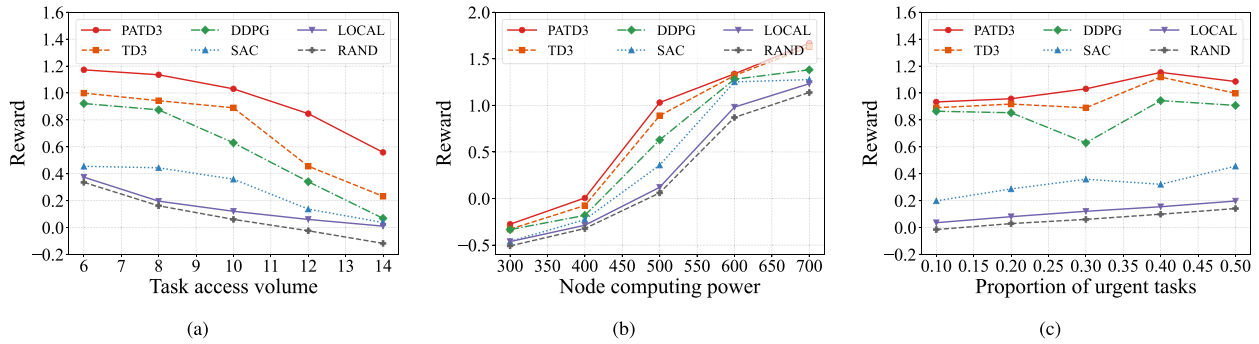


Fig. 7. Rewards in different scenarios: (a) task access volume; (b) node computing power; (c) proportion of urgent tasks.

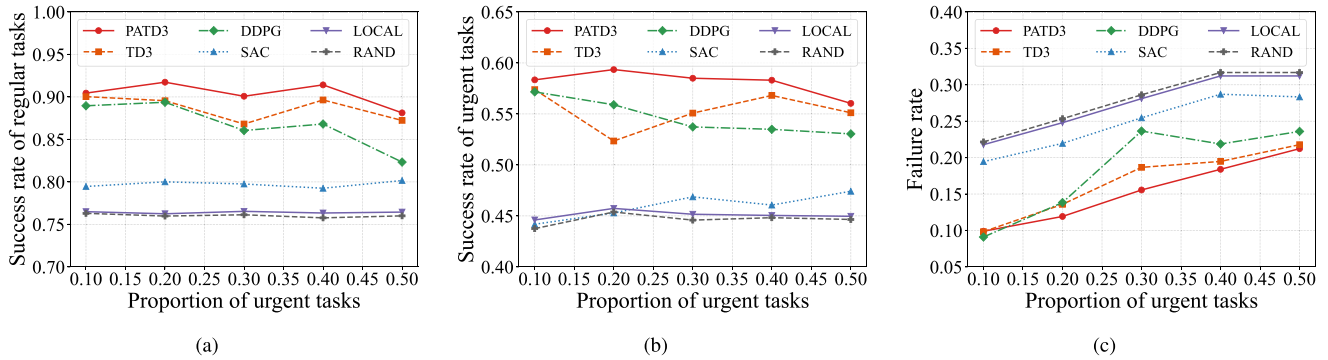


Fig. 8. Task status in different urgent task proportions: (a) regular tasks; (b) urgent tasks; (c) all tasks.

test range increased the proportion of urgent tasks from 0.1 to 0.5. Fig. 7(c) illustrates the reward values of different algorithms as the proportion of urgent tasks changes. The results indicate that as the proportion of urgent tasks increases, the overall reward values exhibit an upward trend. This is primarily due to the growing contribution of successfully executed urgent tasks, which effectively reduces overall delay. Although the overall success rate shows a slight decline, the positive impact of executing more urgent tasks offsets this trend, resulting in higher overall rewards. In addition, Fig. 8(a), (b), and (c) present the success rates for regular tasks, the success rates for urgent tasks, and the failure rates for all tasks, respectively. As the proportion of urgent tasks increases, the success rates of the SAC, LOCAL, and RAND algorithms remain relatively stable. However, their failure rates rise significantly. The PATD3 algorithm, leveraging its priority planning mechanism, effectively ensures the execution of a limited number of urgent tasks and consistently achieves the highest success rates. In contrast, the success rates of the TD3 and DDPG algorithms are less stable, reflecting their inability to adapt effectively to delay-sensitive scenarios. These results highlight the superior adaptability and performance of PATD3 in scenarios with stringent delay requirements, making it a robust choice for handling delay-sensitive tasks.

VII. CONCLUSION

In this work, we proposed a flexible task scheduling policy to enable cooperation among heterogeneous CPN nodes. Building

on the introduced computing system, we developed a priority-aware task scheduling scheme that optimizes decision-making based on the current system status. By constraining the number of tasks scheduled per time slot, the scheme achieves optimal task-to-resource matching. Moreover, incorporating task priority evaluation significantly improves the success rate of task processing within the system. The numerical results demonstrate that the proposed PADRL algorithm outperforms baseline algorithms across multiple dimensions. It enhances the global task success rate, reduces task processing delays, and ensures efficient and balanced resource utilization.

REFERENCES

- [1] Y. Peng et al., "Computing and communication cost-aware service migration enabled by transfer reinforcement learning for dynamic vehicular edge computing networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 1, pp. 257–269, Jan. 2024.
- [2] T. X. Tran, A. Hajisami, P. Pandey, and D. Pompili, "Collaborative mobile edge computing in 5 G networks: New paradigms, scenarios, and challenges," *IEEE Commun. Mag.*, vol. 55, no. 4, pp. 54–61, Apr. 2017.
- [3] T. Liu, S. Ni, X. Li, Y. Zhu, L. Kong, and Y. Yang, "Deep reinforcement learning based approach for online service placement and computation resource allocation in edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 7, pp. 3870–3881, Jul. 2023.
- [4] Q. Tang, F. R. Yu, R. Xie, A. Boukerche, T. Huang, and Y. Liu, "Internet of intelligence: A survey on the enabling technologies, applications, and challenges," *IEEE Commun. Surveys Tut.*, vol. 24, no. 3, pp. 1394–1434, thirdquarter 2022.
- [5] D. Shi, H. Gao, L. Wang, M. Pan, Z. Han, and H. V. Poor, "Mean field game guided deep reinforcement learning for task placement in cooperative multiaccess edge computing," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 9330–9340, Oct. 2020.

- [6] C. Yi, J. Cai, T. Zhang, K. Zhu, B. Chen, and Q. Wu, "Workload re-allocation for edge computing with server collaboration: A cooperative queueing game approach," *IEEE Trans. Mob. Comput.*, vol. 22, no. 5, pp. 3095–3111, May 2023.
- [7] C. Sun, X. Wu, X. Li, Q. Fan, J. Wen, and V. C. Leung, "Cooperative computation offloading for multi-access edge computing in 6 G mobile networks via soft actor critic," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 6, pp. 5601–5614, Nov./Dec. 2024.
- [8] "Y.2501: Computing power network - framework and architecture," Sep. 2021.
- [9] R. Xie et al., "Delay-prioritized and reliable task scheduling with long-term load balancing in computing power networks," *IEEE Trans. Serv. Comput.*, vol. 17, no. 6, pp. 3359–3372, Nov./Dec. 2024.
- [10] X. Tang et al., "Computing power network: The architecture of convergence of computing and networking towards 6 G requirement," *China Commun.*, vol. 18, pp. 175–185, Feb. 2021.
- [11] B. Lei, Q. Zhao, and J. Mei, "Computing power network: An interworking architecture of computing and network based on IP extension," in *Proc. IEEE Int. Conf. High Perform. Switching Routing*, 2021, pp. 1–6.
- [12] R. Pang, H. Li, Y. Ji, G. Wang, and C. Cao, "Energy-saving mechanism based on tidal characteristic in computing power network," in *Proc. Int. Conf. Netw. Netw. Appl.*, Oct. 2021, pp. 150–154.
- [13] J. Liu et al., "Computing power network: A testbed and applications with edge intelligence," in *Proc. IEEE Conf. Comput. Commun. Workshops*, May 2022, pp. 1–2.
- [14] A. Ghose, S. Singh, V. Kulaharia, L. Dokara, S. Maity, and S. Dey, "PySchedCL: Leveraging concurrency in heterogeneous data-parallel systems," *IEEE Trans. Comput.*, vol. 71, no. 9, pp. 2234–2247, Sep. 2022.
- [15] K. Qian, Y. Liu, C. Shu, Y. Sun, and K. Wang, "Fine-grained benchmarking and targeted optimization: Enabling green IoT-oriented blockchain in the 6G era," *IEEE Trans. Green Commun. Netw.*, vol. 7, no. 2, pp. 1036–1051, Jun. 2023.
- [16] C. Liu and P. Wang, "A concurrent message processing mechanism for Internet of Vehicles," in *Proc. Int. Conf. Inf. Technol. Big Data Artif. Intell.*, Dec. 2021, pp. 778–783.
- [17] S. Kekki et al., "MEC in 5G networks," *ETSI White Paper*, vol. 28, pp. 1–28, Jun. 2018.
- [18] X. Wang, J. Ye, and J. C. Lui, "Online learning aided decentralized multi-user task offloading for mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 23, no. 4, pp. 3328–3342, Apr. 2024.
- [19] J. Wu, Z. Cao, Y. Zhang, and X. Zhang, "Edge-cloud collaborative computation offloading model based on improved partial swarm optimization in MEC," in *Proc. Int. Conf. Parallel Distrib. Syst.*, Dec. 2019, pp. 959–962.
- [20] K.-H. Liu, Y.-H. Hsu, W.-N. Lin, and W. Liao, "Fine-grained offloading for multi-access edge computing with actor-critic federated learning," in *Proc. IEEE Wireless Commun. Netw. Conf.*, Mar. 2021, pp. 1–6.
- [21] P. Dai, K. Hu, X. Wu, H. Xing, and Z. Yu, "Asynchronous deep reinforcement learning for data-driven task offloading in mec-empowered vehicular networks," in *Proc. IEEE Conf. Comput. Commun.*, May 2021, pp. 1–10.
- [22] Q. Wang, Y. Mao, Y. Wang, and L. Wang, "Computation tasks offloading scheme based on multi-cloudlet collaboration for edge computing," in *Proc. 7th Int. Conf. Adv. Cloud Big Data*, Sep. 2019, pp. 339–344.
- [23] S. Huang, C. Yang, S. Yin, Z. Zhang, and Y. Chu, "Latency-aware task peer offloading on overloaded server in multi-access edge computing system interconnected by metro optical networks," *J. Lightw. Technol.*, vol. 38, pp. 5949–5961, Nov. 2020.
- [24] L.-T. Hsieh, H. Liu, Y. Guo, and R. Gazda, "Task management for cooperative mobile edge computing," in *Proc. IEEE/ACM Symp. Edge Comput.*, Nov. 2020, pp. 352–357.
- [25] F. Guo, H. Zhang, H. Ji, X. Li, and V. C. M. Leung, "Energy efficient computation offloading for multi-access MEC enabled small cell networks," in *Proc. IEEE Int. Conf. Commun. Workshops*, May 2018, pp. 1–6.
- [26] K. Nakamura et al., "A lora-based protocol for connecting IoT edge computing nodes to provide small-data-based services," *Digit. Commun. Netw.*, vol. 8, pp. 257–266, Sep. 2022.
- [27] Z. Fei, Y. Wang, X. Qin, J. Zhao, and X. Wang, "Joint wireless and computational resources allocation for optimizing the age of correlated information in fog computing networks," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 1, pp. 326–339, Jan./Feb. 2024.
- [28] R. W. Floyd, "Algorithm 97: Shortest path," *Commun. ACM*, vol. 5, no. 6, 1962, Art. no. 345.
- [29] Q. Tang et al., "Joint service deployment and task scheduling for satellite edge computing: A two-timescale hierarchical approach," *IEEE J. Sel. Areas Commun.*, vol. 42, no. 5, pp. 1063–1079, May 2024.
- [30] Y. Ren, R. Xie, F. R. Yu, T. Huang, and Y. Liu, "Quantum collective learning and many-to-many matching game in the metaverse for connected and autonomous vehicles," *IEEE Trans. Veh. Technol.*, vol. 71, no. 11, pp. 12128–12139, Nov. 2022.
- [31] Q. Tang et al., "Collective deep reinforcement learning for intelligence sharing in the internet of intelligence-empowered edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 11, pp. 6327–6342, Nov. 2023.
- [32] F. Jiang, L. Dong, K. Wang, K. Yang, and C. Pan, "Distributed resource scheduling for large-scale MEC systems: A multi-agent ensemble deep reinforcement learning with imitation acceleration," *IEEE Internet Things J.*, vol. 9, no. 9, pp. 6597–6610, May 2022.
- [33] H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. 13th AAAI Conf. Artif. Intell.*, 2016, pp. 2094–2100.
- [34] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [35] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. Int. Conf. Mach. Learn.*, Stockholm, Sweden, Jun. 2018, pp. 1861–1870.
- [36] V. Mnih, "Asynchronous methods for deep reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1928–1937.
- [37] S. Fujimoto, H. Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proc. Int. Conf. Mach. Learn.*, Stockholm, Sweden, Jun. 2018, pp. 1587–1596.
- [38] R. Hashemi, S. Ali, N. H. Mahmood, and M. Latva-Aho, "Deep reinforcement learning for practical phase-shift optimization in RIS-aided MISO URLLC systems," *IEEE Internet Things J.*, vol. 10, no. 10, pp. 8931–8943, May 2023.
- [39] C. Xu, P. Zhang, X. Xia, L. Kong, P. Zeng, and H. Yu, "Digital twin-assisted intelligent secure task offloading and caching in blockchain-based vehicular edge computing networks," *IEEE Internet Things J.*, vol. 12, no. 4, pp. 4128–4143, Feb. 2025.
- [40] Q. Zhang, C. Zhang, J. Zhao, D. Wang, and W. Xu, "Dynamic resource allocation for multi-access edge computing in urban rail transit," *IEEE Trans. Veh. Technol.*, vol. 74, no. 2, pp. 3296–3310, Feb. 2025.
- [41] B. Adhikari, A. S. Khwaja, M. Jaseemuddin, A. Anpalagan, and A. Nallanathan, "Energy efficient RIS-assisted UAV networks using twin delayed DDPG technique," *IEEE Trans. Wireless Commun.*, vol. 23, no. 12, pp. 18423–18439, Dec. 2024.
- [42] Q. Wang et al., "Energy-efficient task split and resource allocation in LEO-satellite-assisted IoT network," *IEEE Internet Things J.*, vol. 11, no. 21, pp. 34519–34527, Nov. 2024.
- [43] B. Hazarika, K. Singh, S. Biswas, S. Mumtaz, and C.-P. Li, "Multi-agent DRL-based task offloading in multiple RIS-aided IoT networks," *IEEE Trans. Veh. Technol.*, vol. 73, no. 1, pp. 1175–1190, Jan. 2024.
- [44] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller, "Deterministic policy gradient algorithms," in *Proc. Int. Conf. Mach. Learn.*, 2014, pp. 387–395.
- [45] Z. Xu et al., "Experience-driven networking: A deep reinforcement learning based approach," in *Proc. IEEE Conf. Comput. Commun.*, 2018, pp. 1871–1879.



Renchao Xie (Senior Member, IEEE) received the Ph.D. degree in electrical engineering from the Beijing University of Posts and Telecommunications, Beijing, China, in 2012. From 2010 to 2011, he visited Carleton University, Ottawa, ON, Canada, as a Visiting Scholar. He is currently, a Professor with the School of Information and Communication Engineering, Beijing University of Posts and Telecommunications. He has authored or coauthored more than 80 papers in reputable journals/conferences, five books, and 41 granted patents. His research interests include edge computing, computing power networks, and industrial internet of things. He was the recipient of the Best Paper Awards at IEEE ICC 2014 and ICC 2022, respectively. He was the Technical Program Committee (TPC) Co-Chair of numerous conferences. He was also a reviewer for several journals, including IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE TRANSACTIONS ON SERVICES COMPUTING, and IEEE TRANSACTIONS ON COMMUNICATIONS.



Li Feng (Graduate Student Member, IEEE) received the B.S. degree in 2022 from the Beijing University of Posts and Telecommunications, Beijing, China, where she is currently working toward the Ph.D. degree. Her research interests include edge/cloud computing and task scheduling.



Ran Zhang (Member, IEEE) received the Ph.D. degree from the Beijing University of Posts and Telecommunications (BUPT), Beijing, China, in 2021. He is currently an Associate Researcher with the School of Information and Communication Engineering, BUPT. His research interests include satellite communications, named data networking, and artificial intelligence.



Qinqin Tang received the Ph.D. degree from the Beijing University of Posts and Telecommunications, Beijing, China, in 2022. She is currently a Postdoctoral Research Fellow with the Beijing University of Posts and Telecommunications. Her research interests include internet of intelligence, edge/cloud computing, traffic offloading, and resource management.



F. Richard Yu (Fellow, IEEE) received the Ph.D. degree in electrical engineering from the University of British Columbia Vancouver, BC, Canada, in 2003. His research interests include connected/autonomous vehicles, artificial intelligence, cybersecurity, and wireless systems. He has been named in the Clarivate Analytics list of “Highly Cited Researchers” since 2019. He was the recipient of the Best Paper Awards from some first-tier conferences. He is an elected member of the Board of Governors of the IEEE VTS and the Editor-in-Chief of IEEE VTS Mobile World newsletter. He is a Fellow of the Canadian Academy of Engineering (CAE), Engineering Institute of Canada (EIC), and IET. He is a Distinguished Lecturer of IEEE in VTS and ComSoc.



Han Zhu received the B.S. degree in electronic science and technology from the Beijing University of Posts and Telecommunications (BUPT), Beijing, China, in 2020, and the M.S. degree from the State Key Laboratory of Networking and Switching Technology, BUPT, in 2023.



Zehui Xiong (Senior Member, IEEE) received the Ph.D. degree from Nanyang Technological University (NTU), Singapore. He is currently an Assistant Professor with the Singapore University of Technology and Design, Singapore, and also an Honorary Adjunct Senior Research Scientist with Alibaba-NTU Singapore Joint Research Institute, Singapore. He was the Visiting Scholar with Princeton University, Princeton, NJ, U.S. and the University of Waterloo, Waterloo, ON, Canada. He has authored or coauthored more than 200 research papers in leading journals and flagship conferences and many of them are ESI Highly Cited Papers. His research interests include wireless communications, Internet of Things, blockchain, edge intelligence, and metaverse. He has won more than 10 Best Paper Awards in International conferences and is listed in the World’s Top 2% scientists identified by Stanford University. He is the Editor or Guest Editor for many leading journals including IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS, IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY, IEEE INTERNET OF THINGS JOURNAL.



Tao Huang (Senior Member, IEEE) received the B.S. degree in communication engineering from Nankai University, Tianjin, China, in 2002, and the M.S. and Ph.D. degrees in communication and information system from the Beijing University of Posts and Telecommunications, Beijing, China, in 2004 and 2007, respectively. He is currently a Professor with the Beijing University of Posts and Telecommunications. His research interests include network architecture, network artificial intelligence, and network virtualization.